Testing Report: Custom System Calls

Group 4
Assignment 2

Name	Roll Number
Payidiparthy Sri Surya Naga Hadwik	23CS10052
Chintada Yesheeth Sree Narayana	23CS10013
Dasari Veera Venkata Abhinav Teja	23CS10017

Environment

- **OS:** Ubuntu 24.04.4 LTS (running in VirtualBox)
- **Kernel:** Linux 6.1.6 (compiled from source)
- **Compiler:** GCC (Ubuntu default)
- **Architecture:** x86_64

Task A: Process Information (`proc_info`)

Test 1: Current Process Info (PID Mode)

Input: `get_proc_info(getpid())`
Expected: Returns valid process information for the running test process.
Result: PASS

```
Parent PID: 3616
State: 0 (TASK_RUNNING)
Priority: 120 (default for SCHED_NORMAL)
Scheduling Class: SCHED_NORMAL
Child Processes: 0
Sibling Processes: 1
```

Verification: State 0 (RUNNING) is correct since the process is actively executing the syscall. Priority 120 is the default for normal processes. SCHED_NORMAL is the expected scheduler for user programs.

Test 2: Init Process (PID 1)

Input: `get_proc_info(1)`
Expected: Returns information about the init/systemd process.
Result: PASS

```
Parent PID: 0 (init's parent is the kernel)
State: 1 (TASK_INTERRUPTIBLE)
```

```
Priority: 120
Scheduling Class: SCHED_NORMAL
Child Processes: 33
Sibling Processes: 2
```

Verification: PID 1 (systemd) correctly shows parent PID 0 (the kernel idle process). The high child count reflects the system services spawned by init.

Test 3: System-Wide Info

Input: `get_system_info()`
Expected: Returns aggregate system statistics with consistent process counts.
Result: PASS

```
Total Processes: 327
Running: 1
Interruptible: 219
Uninterruptible: 0
RT Class: 57
Fair Class: 270
In CFS Runqueue: 1
Min Vruntime PID: 4436
Min Vruntime: 48932457838
Total CFS Load: 1048576
```

Verification: RT (57) + Fair (270) = 327 = Total Processes, confirming counts are consistent. Only 1 process is running (the test program itself), which is expected on a mostly idle system. The min vruntime PID matches the test program's PID.

Test 4: Invalid PID

Input: `syscall(451, 99999, PROC_INFO_PID, buf, sizeof(buf))`
Expected: Returns -1 with `errno` set to `ESRCH`.
Result: PASS — `Error: No such process`

Test 5: Invalid Flags

Input: `syscall(451, 1, 99, buf, sizeof(buf))`
Expected: Returns -1 with `errno` set to `EINVAL`.
Result: PASS — `Error: Invalid argument`

Test 6: Buffer Too Small

Input: `syscall(451, 1, PROC_INFO_PID, buf, 1)`
Expected: Returns -1 with `errno` set to `ENOSPC`.
Result: PASS — `Error: No space left on device`

Task B: Context Switch Tracker

Test 1: Register Current Process

Input: `register_pid(getpid())`
Expected: Returns 0, process is added to the monitored list.
Result: PASS — `Registered PID 3859 successfully`

Test 2: Register PID 1 (systemd)

Input: `register_pid(1)`
Expected: Returns 0, PID 1 is added to the monitored list.
Result: PASS — `Registered PID 1 successfully`

Test 3: Register Child Processes

Input: Two child processes spawned via `fork()`; both registered with `register_pid()`.
Expected: Returns 0 for both, children are added to the monitored list.
Result: PASS — `Registered PID 3860 successfully`, `Registered PID 3861 successfully`

Test 4: Fetch with 4 Processes Monitored

Input: `fetch_ctx_switches(&stats)` with PIDs 3859, 1, 3860, 3861 registered.
Expected: Returns cumulative context switches across all four processes and their threads.
Result: PASS

```
Voluntary Context Switches: 65097
Involuntary Context Switches: 2523
```

Verification: The high voluntary count is largely contributed by PID 1 (systemd), which frequently sleeps waiting for events. The child processes contribute additional switches from their busy work and `usleep()` calls.

Test 5: Deregister Child 1

Input: `deregister_pid(3860)`
Expected: Returns 0, child is removed from the list.
Result: PASS — `Deregistered PID 3860 successfully`

Test 6: Fetch with 3 Processes Monitored

Input: `fetch_ctx_switches(&stats)` with PIDs 3859, 1, 3861 remaining.
Expected: Lower cumulative count after removing one child.
Result: PASS

```
Voluntary Context Switches: 65086
Involuntary Context Switches: 2520
```

Verification: Both counts decreased compared to Test 4 (voluntary: 65097 $\rightarrow$ 65086, involuntary: 2523 $\rightarrow$ 2520), confirming PID 3860's contribution was removed.

Test 7: Deregister Child 2

Input: `deregister_pid(3861)`
Expected: Returns 0, child is removed from the list.
Result: PASS — `Deregistered PID 3861 successfully`

Test 8: Fetch with 2 Processes Monitored

Input: `fetch_ctx_switches(&stats)` with PIDs 3859 and 1 remaining.
Expected: Further decrease after removing second child.
Result: PASS

```
Voluntary Context Switches: 65075
Involuntary Context Switches: 2519
```

Verification: Counts decreased again (voluntary: 65086 → 65075, involuntary: 2520 → 2519), confirming PID 3861's contribution was removed.

Test 9: Deregister Current Process

Input: `deregister_pid(3859)`
Expected: Returns 0, current process is removed from the list.
Result: PASS — `Deregistered PID 3859 successfully`

Test 10: Fetch with Only PID 1

Input: `fetch_ctx_switches(&stats)` with only PID 1 remaining.
Expected: Returns context switches for PID 1 alone.
Result: PASS

```
Voluntary Context Switches: 65074
Involuntary Context Switches: 2519
```

Verification: Voluntary count dropped by 1 (65075 → 65074), confirming the test process was properly removed. The clear downward trend across Tests 4, 6, 8, and 10 validates that `deregister` correctly removes processes from the monitored list.

Test 11: Deregister PID 1

Input: `deregister_pid(1)`
Expected: Returns 0, PID 1 is removed.
Result: PASS — `Deregistered PID 1 successfully`

Test 12: Invalid PID

Input: `register_pid(-1)`
Expected: Returns -1 with `errno` set to `EINVAL`.
Result: PASS — `register_pid failed: Invalid argument`

Test 13: Deregister Non-Existent PID

Input: `deregister_pid(99999)`
Expected: Returns -1 with `errno` set to `ESRCH`.
Result: PASS — `deregister failed: No such process`

Test 14: Double Deregister

Input: `deregister_pid(3859)` (already deregistered in Test 9).
Expected: Returns -1 with `errno` set to `ESRCH`.
Result: PASS — `deregister failed: No such process`

Verification: Confirms that a PID cannot be deregistered twice, and the list correctly reflects prior removals.

Summary

Task	Test Case	Result
A	Process-specific mode (current process)	PASS
A	Process-specific mode (PID 1)	PASS
A	System-wide mode	PASS
A	Invalid PID → ESRCH	PASS
A	Invalid flags → EINVAL	PASS
A	Buffer too small → ENOSPC	PASS
B	Register current process	PASS
B	Register PID 1	PASS
B	Register child processes	PASS
B	Fetch (4 processes monitored)	PASS
B	Deregister child 1, fetch (3 monitored)	PASS
B	Deregister child 2, fetch (2 monitored)	PASS
B	Deregister self, fetch (1 monitored)	PASS
B	Deregister PID 1	PASS
B	Invalid PID → EINVAL	PASS
B	Deregister non-existent PID → ESRCH	PASS
B	Double deregister → ESRCH	PASS

All 17 tests passed.